



HOW TO INSTALL RTAI

IN BECKHOFF CX1020

v.0.1

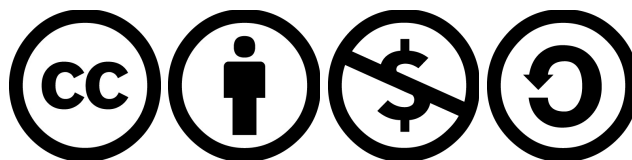
Authors: J. Andrés Cubillos Galvis

Ekaitz Larzabal Balerdi

Mikel Larrea Sukia

Eloy Irigoyen Gordo

Date:09/05/12



This work is licensed under the Creative Commons Atribución-NoComercial-CompartirIgual 3.0 Unported License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-nc-sa/3.0/> or send a letter to Creative Commons, 444 Castro Street, Suite 900, Mountain View, California, 94041, USA.

Contents

GENERAL INFORMATION.....	4
PRELIMINARIES.....	5
STEP 1 : INSTALLATION OF XUBUNTU, AND NETWORK SETUP.....	7
STEP 2 : PACKAGES UPDATE FOR RTAI INSTALLATION.....	10
STEP 3 : DOWNLOAD AND UNCOMPRESS RTAI.....	11
STEP 4 : DOWNLOAD, UNCOMPRESS AND PATCH THE NEW KERNEL.....	12
STEP 5 : CONFIGURE AND COMPILE THE NEW KERNEL.....	13
STEP 6 : MESA AND eFLTK LIBRARIES INSTALLATION.....	15
STEP 7 : CONFIGURATION AND INSTALLATION OF RTAI.....	17
STEP 8 : TEST INSTALLATION OF RTAI.....	18
STEP 9 : DOWNLOAD AND INSTALLATION OF COMEDI AND COMEDILIB.....	19
STEP 10 : SCRIPT FOR THE CREATION OF INODES FOR RTAI AND COMEDI.....	21
STEP 11 : CODE GENERATION ORIENTED CONFIGURATION FOR RTAI USING MATLAB® /SIMULINK®.....	23
STEP 12 : CODE GENERATION EXAMPLE.....	25
AUTHOR'S NOTE.....	33

GENERAL INFORMATION

1. **Description:** This "How to" comprises the installation of Linux (Xubuntu 8.04) / RTAI (3.8) in CX1020 unit of Beckhoff, the code generation from the PC (Host) with Matlab® / Simulink®, and the subsequent compilation and execution CX1020 unit with RTAI.
2. **Sources / References:**

STEP BY STEP RTAI TARGET FOR BEGINNERS
Giampiero Campa, © 2009 The MathWorks, Inc.
<https://www.rtai.org/RTAILAB/RTAI-TARGET-HOWTO.txt>

RTAI-Lab tutorial: Scilab, Comedi, and real-time control
Roberto Bucher
<https://www.rtai.org/RTAILAB/RTAI-Lab-tutorial.pdf>

RTAI 3.4 User Manual rev 0.3
<https://www.rtai.org> ; Documentation / Reference Documents

Technical Report: How to install an RTAI extension in Linux 2.6.24 Kernel
Pedro Henrique Santana
http://www.lara.unb.br/~phsantana/data/files/technotes/HowTo_RTAI_Linux.pdf

IMPORTANT:

USE THIS INFORMATION UNDER YOUR OWN RISK. THE GICI IS NOT RESPONSIBLE FOR ANY DAMAGE OR DEGRADATION IN THE EQUIPMENTS OR DEVICES, NEITHER FOR DAMAGE THAT THEY CAN PRODUCE A THIRD PARTY.

PRELIMINARIES

1. Hardware:

CX-1020-0121	→	CPU (Intel® Celeron® M ULV 1 Ghz) 2 RJ45 Sockets 2 USB ports + 1 DVI port 1GB RAM (Modified, factory settings 256 MB).
CX-1100-0002	→	Power supply for CPU and terminals E/S K-Bus.
KL3404	→	Analog Inputs Terminal, 4 channels, +/- 10 V.
KL4034	→	Analog Outputs Terminal, 4 channels, +/- 10 V.
Compact Flash	→	8.0 GB.

2. Connection accessories and complementary equipments:

Power Supply 24V (output)

USB-CD reader

USB (Hub)

Keyboard and Mouse, both USB

Display

3. Get the ISO of Xubuntu 8.04.1 (Hardy Heron), and burn it on a CD

ISO available in: <http://cdimage.ubuntu.com/xubuntu/releases/hardy/release/>

File: xubuntu-8.04.1-desktop-i386.iso

4. Connect on the CX1020 unit:

Keyboard, Mouse and USB-CD reader

Note: Since the CX1020 only has 2 USB ports, it's recommendable using a USB Hub.

5. Insert the Compact Flash (CF) 8.0 GB within their respective slot. (Preferably formatted)

6. Turn on the unit CX-1020, and enter to the BIOS setup (pressing key "DEL o Supr" at start-up).

7. In the BIOS, in "Advanced BIOS Features", modify the following values, leave the others as default:

First Boot Device	[USB-CDROM]
Second Boot Device	[Disabled]
Third Boot Device	[Disabled]
Boot Other Device	[Disabled]

8. Insert the CD installation of Xubuntu in USB-CD reader.

9. Save the changes and exit from the BIOS.

Press	F10
Write	“Y”
Press	Enter

System restarts

STEP 1 : INSTALLATION OF XUBUNTU, AND NETWORK SETUP

Note: The values selected in the setup option below are chosen by the author. However according your preferences and own experience, these values can be changed.

1. Choose the installation language

English

2. Appears a menu with Xubuntu logo

Select "Install Xubuntu" + Enter

3. Installation of Xubuntu

- 3.1. Select the installation language (Step 1 de 7)

English

- 3.2. Select the Time Zone (Step 2 de 7)

Madrid

- 3.3. Select keyboard layout (Step 3 de 7)

Spain / Spain

- 3.4. Select Disk Partitions (Step 4 de 7)

Guided – use entire disk

|

+ - SCSI1 (0,0,0) - 8.0 GC ATA ELITE PRO CF CAR

- 3.5. Set name and password user (Step 5 de 7)

- What is your name?: "user_name"
- What name do you want to use to log in?: "user_name"
- Choose a password to keep your account safe: *****
- What is the name of this computer?: "equipment_name"

- 3.6. Ready to install (Step 7 de 7). Is shown a summary of the selected choices.

Click on "Install".

- 3.7. At the end of installation, the system restarting is required. Remove the CD and press "Enter"

System restarts automatically. While the system restarts, enter in the BIOS (pressing "Del"), and choose HDD-0 like first boot device.

- 3.8. In the BIOS, in “Advanced BIOS Features”, modify the following values and leave the others as default:

First Boot Device	[HDD-0]
Second Boot Device	[Disabled]
Third Boot Device	[Disabled]
Boot Other Device	[Disabled]

- 3.9. Save the changes and exit from the BIOS

Press	F10
Write	“Y”
Press	Enter

- 3.10. Enter to Xubuntu.

4. Network Setup:

Following steps are required to download several packages for RTAI installation.

There are different options to access the network:

- Connect directly to the network: DHCP-IP dynamic. **Note:** If you have this option jump to STEP 2: UPDATE PACKAGES TO INSTALLATION OF RTAI.
- Connect directly to the network: IP static. The following instructions are for this kind of setup.

- 4.1. Open a terminal from “Applications / Accesories / Terminal “

- 4.2. Plug the network wire to the CX1020 unit, inside any socket of the CX1020-N000 module.

Note: The MAC for eth0 link is intended for EtherCAT (internal). The MAC for eth1 link is intended for ethernet. The two RJ-45 sockets of the CX1020-N000 module are internally connected to one switch, which is associated to the MAC of eth1. Thus, both connectors point out the same address.

- 4.3. Setup DNS servers

- a) Open the file /etc/resolv.conf with the text editor preferred (in this case “Nano”)

```
$ sudo nano /etc/resolv.conf
```

- b) Once the editor is opened, add the available addresses of DNS server.

```
nameserver ***.***.***.***      (Server addresses)
nameserver ***.***.***.***
```

- c) Save the changes and exit (for "Nano"):

```
Ctrl + x
“Y”
"Enter"
```


4.4. Setup the IP:

- a) Open the file “interfaces“ placed in “ /etc/network “

```
$ sudo nano /etc/network/interfaces
```

- b) Once the editor is opened, look for the eth1 link, whether exist it or not, it has to be edited as follows (leave the rest of the file intact):

```
auto eth1
iface eth1 inet static
    address ***.***.***.*** (Set values according to the network configuration)
    netmask ***.***.***.***
    gateway ***.***.***.***
```

- c) Save the changes and exit (for "Nano"):

```
Ctrl + x
"Y"
"Enter"
```

4.5. Update the changes

```
$ sudo /etc/init.d/networking restart
```

The screen displays:

```
* Reconfiguring network interfaces.... [ OK ]
```

4.6. Verify the link setup:

```
$ ifconfig
```

Existing links are displayed with their respective settings.

STEP 2 : PACKAGES UPDATE FOR RTAI INSTALLATION

1. Open a terminal:

Applications / Accessories / Terminal

2. Some commands require super-user permission. Login as super-user:

For instance: “user_name” = user ; “equipment_name” = beckhoff

```
user@beckhoff:~$ sudo -s
```

```
[sudo] password for user: ***** (Write password user)
```

```
root@beckhoff:~#
```

Onwards, it is assumed that (\$) representing the user and (#) to the super-user.

3. Install the following packages.

General use:

```
# apt-get update
# apt-get install build-essential
```

Linux kernel:

```
# apt-get install kernel-package
# apt-get install libncurses5-dev
```

RTAI-Lab:

```
# apt-get install subversion      [Mesa]
# apt-get install autoconf
# apt-get install automake
# apt-get install libxmu-dev      [eFLTK]
# apt-get install libxi-dev      [eFLTK]
```

STEP 3 : DOWNLOAD AND UNCOMPRESS RTAI

1. Go to the directory “ /usr/src “

```
# cd /usr/src
```

2. Download RTAI:

```
# wget --no-check-certificate https://www.rtai.org/RTAI/rtai-3.8.tar.bz2
```

3. Uncompress the file:

```
# tar -xjvf rtai-3.8.tar.bz2
```

4. Create a symbolic link for RTAI:

```
# rm -f rtai  
# ln -fs rtai-3.8 rtai
```

STEP 4 : DOWNLOAD, UNCOMPRESS AND PATCH THE NEW KERNEL

In the directory “ /usr/src “

1. Download the new kernel:

```
# wget http://www.kernel.org/pub/linux/kernel/v2.6/linux-2.6.24.tar.bz2
```

Note: Other kernel versions can be used, as long as RTAI version has a patch for it.
See directory “patches” in:
“ ../rtai3.8/base/arch/****/patches ” (Where [****] is the processor architecture).

2. Uncompress the file:

```
# tar -xjvf linux-2.6.24.tar.bz2
```

3. The above procedure creates the “linux-2.6.24” directory. Rename by “linux-2.6.24-rtai”

```
# mv linux-2.6.24 linux-2.6.24-rtai
```

4. Create a symbolic link for the new kernel:

```
# rm -f linux  
# ln -fs linux-2.6.24-rtai linux
```

5. Go to the directory “linux”

```
# cd linux
```

6. Patch the new kernel:

```
# patch -p1 -b < /usr/src/rtai/base/arch/x86/patches/hal-linux-2.6.24-x86-2.0-07.patch
```

STEP 5 : CONFIGURE AND COMPILE THE NEW KERNEL

In the directory “ /usr/src/linux “

1. Get the current kernel version:

```
# uname -r
2.6.24.19-generic(in this case)
```

2. Make a copy of the current kernel configuration:

```
# cp -f /boot/config-2.6.24-19-generic .config
```

3. Configure the new kernel:

```
# make menuconfig
```

4. Remain until a new configure window is opened with the title:

```
".config - Linux Kernel v2.6.24 Configuration "
```

The following items show the properties that have to change or maintain their state. All the other settings are left to their default value.

- 4.1. General setup →

```
|
+ - - [*] Prompt for development and/or incomplete code/driver [Enabled]
```

- 4.2. [*] Enable loadable module support → [Enabled]

```
|
+ - - [ ] Module versioning support [Disabled]
```

- 4.3. Processor type and features →

```
|
+ - - [ ] Symmetric multi-processing support [Disabled]
```

```
|
+ - - Processor family → Pentium M [Select]
```

```
|
+ - - Preemption Model →
```

```
|
|
+ - - Preemptible Kernel (Low-Latency Desktop) [Select]
```

```
|
+ - - [*] Interrupt pipeline [Enabled]
```

```
|
+ - - High Memory Support → off [Select]
```

- 4.4. Power Management Options →

```
|
+ - - [ ] Power Management Support [Disabled]
```

```
|
+ - - CPU Frequency scaling →
```

```
|
+ - - [ ] CPU Frequency scaling [Disabled]
```

4.5. Kernel Hacking →

|
+ - - [] Compile the kernel with frame pointers **[Disabled]**

4.6. Save the changes and exit.

5. Compile the new kernel: (The process may take a few hours)

```
# make-kpkg clean
```

```
# make-kpkg --append-to-version --rtai --initrd kernel_image kernel_headers kernel_source
```

6. The above procedure generates the following files in “/usr/src “

```
linux-headers-2.6.24-rtai_2.6.24-rtai-10.00.Custom_i386.deb  
linux-image-2.6.24-rtai_2.6.24-rtai-10.00.Custom_i386.deb  
linux-source-2.6.24-rtai_2.6.24-rtai-10.00.Custom_all.deb
```

7. Go to the directory “/usr/src “, and install all packages .deb

```
# cd /usr/src
```

```
# dpkg -i *.deb
```

This procedure updates automatically the “grub”, as well as creates the kernel images, ramdisk and system.map (in /boot directory).

8. Restart the system.

9. Open a terminal and verify that the system works with the “new” kernel:

```
$ uname -r
```

The screen displays:

```
2.6.24-rtai
```

STEP 6 : MESA AND eFLTK LIBRARIES INSTALLATION

Note: Mesa and eFLTK libraries are required to execute RTAI-Lab.

Mesa library version used: 7.5.1

Open a terminal, and log like a super-user

1. Go to the directory “ /usr/local/src ”
`# cd /usr/local/src`
2. Download the Mesa library:
`# wget ftp://ftp.freedesktop.org/pub/mesa/7.5.1/MesaLib-7.5.1.tar.bz2`
3. Uncompress the file:
`# tar -xjvf MesaLib-7.5.1.tar.bz2`
4. Enter into the directory “Mesa-7.5.1”
`# cd Mesa-7.5.1`
5. Execute:
`# make linux-x86`
(This process may takes a while)
`# make install`
6. Restart the system and verify that it starts without problems.
Note: In case of problems see <http://www.mesa3d.org/>
7. Open a terminal and log like super-user again:
8. Go to the directory “ /usr/local/src “
`# cd /usr/local/src`
9. Download the files:
`# svn co https://ede.svn.sourceforge.net/svnroot/ede/trunk/efltk`
`# svn co https://ede.svn.sourceforge.net/svnroot/ede/trunk/ede`
10. Go to the directory “efltk”:
`# cd efltk`

11. Execute:

```
# autoconf

# ./configure --disable-mysql --disable-unixODBC

# ./emake

    .
    (This process may take a while)
    .
# ./emake install
```

12. Edit the file “ld.so.conf” placed in “/etc”:

```
# nano /etc/ld.so.conf
```

Once the editor is opened, go to the last line and add:

```
/usr/local/lib
```

12.1. Save the changes and exit (“Nano”)

```
Ctrl + x
“Y”
"Enter"
```

13. Execute:

```
# /sbin/ldconfig
```

14. Restart the system.

STEP 7 : CONFIGURATION AND INSTALLATION OF RTAI

Open a terminal and login as super-user

1. Go to the directory “ /usr/src/rtai “

```
# cd /usr/src/rtai
```

2. Execute:

```
# make menuconfig
```

3. This opens the configuration menu “ RTAI: RTAI 3.8 (vulcano) Configuration ”

4. Verify the following values (the others are left as default)

- 4.1. General →

```
|  
+ - - Installation directory (/usr/realtime)  
|  
+ - - Linux source tree (/usr/src/linux)
```

- 4.2. Machine (x86) →

```
|  
+ - - Number of CPUs (SMP-only) (1)      [Change to 1, default is 2]
```

- 4.3. Add-ons →

```
|  
+ - - [ ] Real Time COMEDI support in user space [ *Disabled. It will be later]
```

- 4.4. RTAI Lab →

```
|  
+ - - [ ] RTAI Lab [ *Disabled. It will be later]
```

- 4.5. Save the changes and exit

5. Execute:

```
# make
```

```
.  
(This process may take a while)
```

```
# make install
```

6. Restart the system

```
# reboot
```

STEP 8 : TEST INSTALLATION OF RTAI

1. Open a terminal and log like a super-user

For information about the expected results see document:

RTAI 3.4 User Manual rev 0.3 (October 2006)

2. Execute:

```
# cd /usr/realtime/testsuite/user/latency; ./run
```

```
# cd /usr/realtime/testsuite/user/preempt; ./run
```

```
# cd /usr/realtime/testsuite/user/switches; ./run
```

```
# cd /usr/realtime/testsuite/kern/latency; ./run
```

```
# cd /usr/realtime/testsuite/kern/preempt; ./run
```

```
# cd /usr/realtime/testsuite/kern/switches; ./run
```

STEP 9 : DOWNLOAD AND INSTALLATION OF COMEDI AND COMEDILIB

1. Go to the directory “ /usr/src “

```
# cd /usr/src
```

2. Download the files:

```
# wget http://www.comedi.org/download/comedi-0.7.76.tar.gz
```

```
# wget http://www.comedi.org/download/comedilib-0.8.1.tar.gz
```

3. Uncompress the files:

```
# tar -xzf comedi-0.7.76.tar.gz
```

```
# tar -xzf comedilib-0.8.1.tar.gz
```

4. Create a symbolic link for Comedi

```
# rm -f comedi
```

```
# ln -fs comedi-0.7.76 comedi
```

5. Configure and install Comedi and Comedilib. Go to:

```
# cd /usr/src/comedilib-0.8.1
```

6. Execute:

```
# ./configure --sysconfdir=/etc
```

```
# make
```

```
# make install
```

```
# make dev
```

7. Go to the directory “ /usr/src/comedi “

```
# cd /usr/src/comedi
```

8. Execute:

```
# ./configure --with-linuxdir=/usr/src/linux --with-rtaidir=/usr/realtime
```

```
# make
```

```
.(This process may take a while)
```

```
# make install
```

9. Copy the files:

```
# cp include/linux/comedi.h /usr/include/
```

```
# cp include/linux/comedilib.h /usr/include/
```

10. Execute:

```
# mkdir /usr/local/include/linux
```

```
# ln -s /usr/include/comedi.h /usr/local/include/linux/comedi.h
```

```
# ln -s /usr/include/comedilib.h /usr/local/include/linux/comedilib.h
```

11. Restart the system (Everything should go ok :P)

12. Open a terminal, login as super-user and go to the directory “/usr/src/rtai”

```
# cd /usr/src/rtai
```

13. Execute:

```
# make menuconfig
```

14. The “RTAI 3.8 (vulcano) Configuration” menu opens again, and adjust the following options:

- 14.1. Add-ons →

```
|  
+ - - [ * ] Real Time COMEDI support in user space [Enabled]  
|  
+ - - (/usr/local)COMEDI installation directory. (Leave “/usr/local”, default value)
```

- 14.2. RTAI Lab →

```
|  
+ - - (/usr/local) eFLTK installation directory. (Leave “/usr/local”, default value)
```

- 14.3. Save the changes and exit.

15. Compile and install: (These processes may take a while)

```
# make
```

```
# make install
```

16. Restart the system and repeat the Step 8 (Test installation of RTAI).

STEP 10 : SCRIPT FOR THE CREATION OF INODES FOR RTAI AND COMEDI

1. Go to the directory “ /etc/init.d “

```
# cd /etc/init.d
```

2. Create rtai-inodes.sh script with the text editor preferred (in this case “Nano”)

```
# nano rtai-inodes.sh
```

3. Once open the editor, copy the below text:

```
#!/bin/sh
### BEGIN INIT INFO
# Provides:          GICI – EHU
# Required-Start: $remote_fs $syslog
# Required-Stop: $remote_fs $syslog
# Default-Start:     2 3 4 5
# Default-Stop:      0 1 6
# Short-Description: Inodes - RTAI and Comedi
# Description:
#
### END INIT INFO

# Create shared memory inode
rm -f /dev/rtai_shm
mknod -m 666 /dev/rtai_shm c 10 254

# RTAI inode creation for UDEV systems, creates /dev/rtf(n)
rm -f /dev/comedi* /dev/rtf*

for n in `seq 0 9`; do
    rm -f /dev/rtf$n
    mknod -m 666 /dev/rtf$n c 150 $n
done

for i in `seq 0 15`; do
    rm -f /dev/comedi$i
    mknod -m 666 /dev/comedi$i c 98 $i
done
```

4. Save the changes and exit the editor (in “Nano”)

```
Ctrl + x
“Y”
"Enter"
```

5. Assign permissions and update:

```
# chmod 777 rtai-inodes.sh
```

```
# update-rc.d rtai-inodes.sh start 80 S .      (Be careful, remember the point at the end of the instruction)
```

6. Restart the system

```
# reboot
```

7. Open a terminal. To test the proper function of the script go to the directory “ /dev “

```
$ cd /dev
```

```
$ ls rtai* rtf* comedi*
```

(The directories comedi(0-15), rtai_shm, and rtf(0-9) are shown).

STEP 11 : CODE GENERATION ORIENTED CONFIGURATION FOR RTAI USING MATLAB® /SIMULINK®

Preliminaries:

- The developed model has been carried out in a PC (Host) with WinXP® and Matlab ® R2011a + Real-Time Workshop®.
 - The procedure has been: Generate code in Matlab®, then download it in the CX-1020 unit (Target) with Xubuntu / RTAI 3.8, compile and execute it.
 - The file transfer can be done in several ways (PenDrives, Shared Folders, etc). This process uses a PenDrive.
1. Insert a PenDrive into the **CX-1020** (Target) unit and copy in it the directory "matlab" placed in:

“ /usr/src/rtai/rtai-lab “

2. Remove the PenDrive and plug it into the **PC** (Host)
3. In the **PC** (Host), go to the directory:

“ C:\Archivos de programa\MATLAB\R2011a\rtw\c “

4. Create the directory "rtai" and copy all the content in the directory "matlab" of the PenDrive to it.
5. Open Matlab® and execute the file "setup.m". File placed in:

“ C:\Archivos de programa\MATLAB\R2011a\rtw\c\rtai ”.

The execution of this file creates the following elements:

- Files MEX for the s-functions contained in:
- “C:\Archivos de programa\MATLAB\R2011a\rtw\c\rtai\devices”
- A directory with "RTAI Devices" blocks in Simulink®.
6. Edit the file "rtai.tmf" (with Block of notes or other plain text editor) placed in :

Change: MATLAB_ROOT = |>MATLAB_ROOT<|
by: MATLAB_ROOT = /usr/local/matlab

Note: “ /usr/local/matlab ” it will be the “Matlab_root“ in the CX-1020 unit (Target)

Change: COMPUTER = |>COMPUTER<|
by: COMPUTER = GLNX86

Note: This is the architecture of CX-1020 unit

Change: LINUX_HOME = \$(shell rtai-config --linux-dir 2>/dev/null)
by: LINUX_HOME = /usr/src/linux

Change:	RTAIDIR	=	\$(shell rtai-config --prefix 2>/dev/null)
by:	RTAIDIR	=	/usr/realtime
Change:	COMEDI_HOME	=	\$(shell rtai-config --comedi-dir 2>/dev/null)
by:	COMEDI_HOME	=	/usr/src/linux

In the "Rules" section delete the lines:

```
|>START_EXPAND_RULES<|%.o : |>EXPAND_DIR_NAME<|/%.c
    gcc -c $(CFLAGS) $<

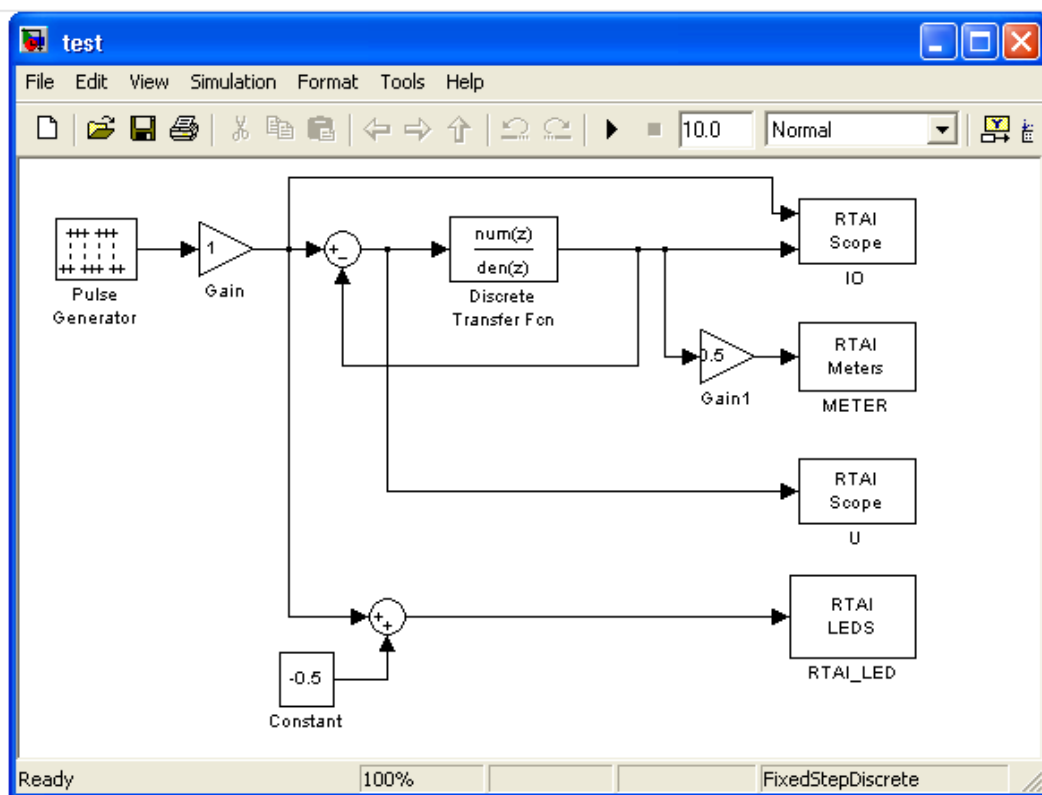
|>END_EXPAND_RULES<|
```

Finally, save the changes and exit.

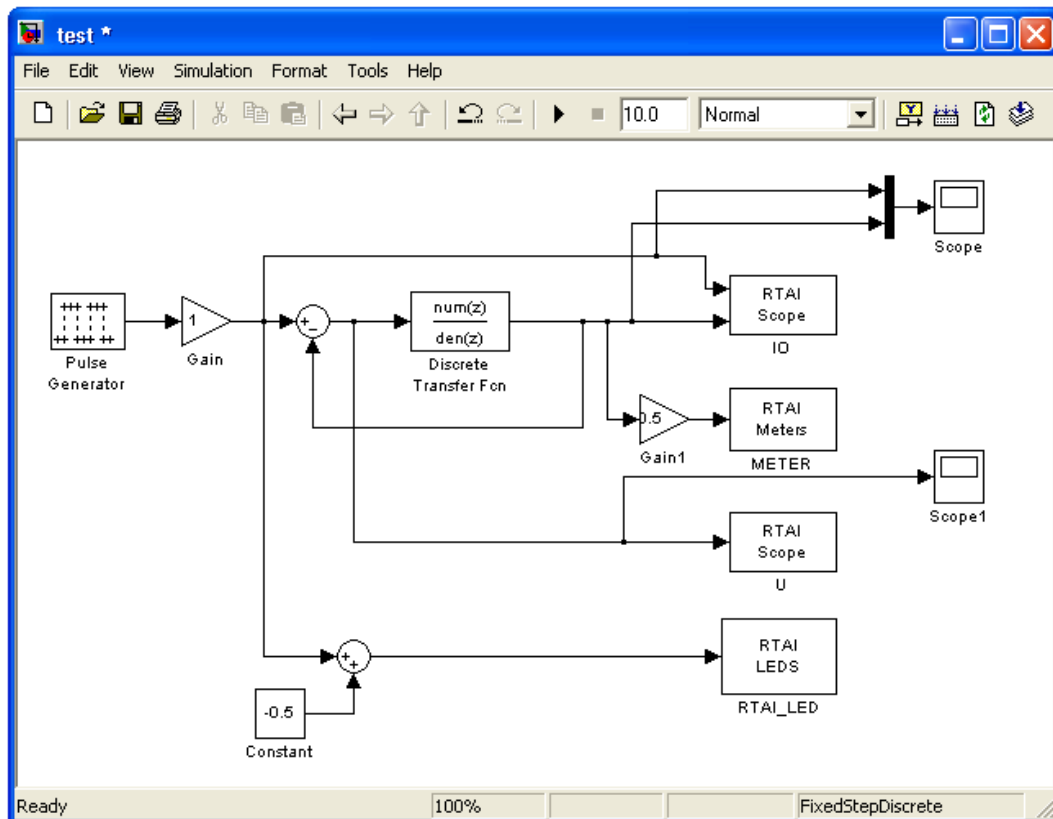
STEP 12 : CODE GENERATION EXAMPLE

1. In the PC (Host) create the directory "ejemplo-rtai" in " [C:\...](#)My Documents\MATLAB "
2. Go to the directory " C:\Program Files\MATLAB\R2011a\rtw\c\rtai\examples "
3. Copy the file "test.mdl"
4. Paste it in the directory " C:\ ...My Documents \MATLAB\ejemplo-rtai " (Created in item 1.)
5. From Matlab® open the file "test.mdl", placed in "C:\ ..My Documents\MATLAB\ejemplo-rtai".

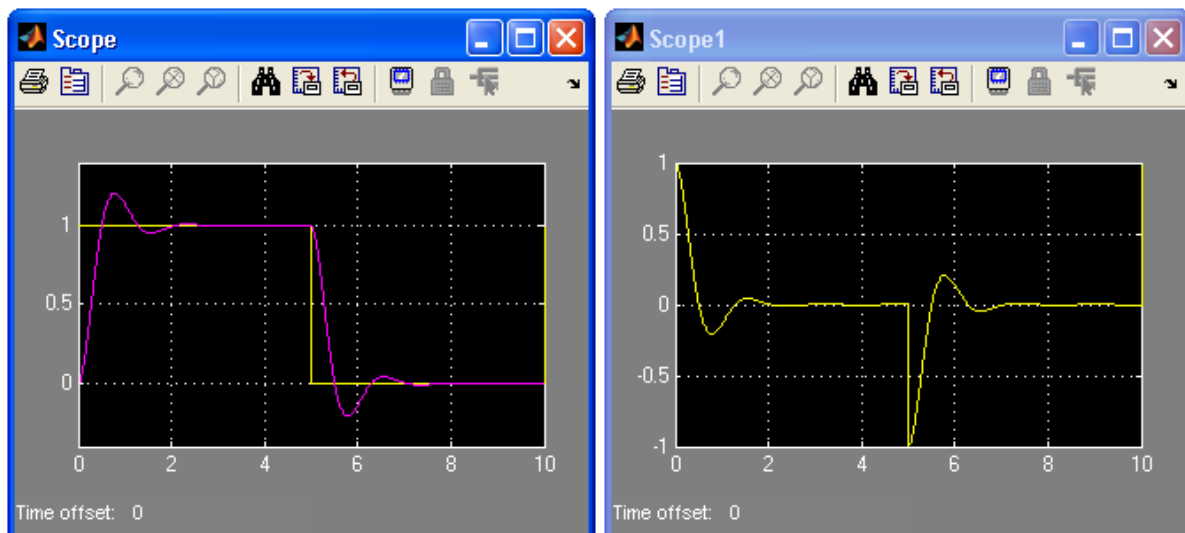
A window opens with this model



Note: To visualize model operation, add the Simulink® block “Scopes”



The system shows the following behavior.



The window "Scope" shows the input (yellow) and output (magenta) of the system. It can be observed that the input signal is a pulse train (period = 10 s / duty cycle = 50%) and the output signal has a Peak value ≈ 1.25 [un] at $\approx 0,7$ [s] (Peak time).

The window "Scope 1" shows the control signal (yellow).

6. In the model window "test" go to "Simulation / Configuration | Parameters" menu.
7. In "Code Generation" set the following values:

Target Selection →

System target file:	rtai.tlc
Language:	C
Description:	Linux RTAI Real-Time Target

Makefile configuration →

Generate makefile:	[OK]
Make command:	make_rtw
Template makefile:	rtai.tmf

8. Generate code by clicking in "Build"

When developing a special functions on Matlab®/Simulink® take into account the note of Giampiero Campa in STEP BY STEP RTAI TARGET FOR BEGINNERS © 2009:

"If your simulink scheme uses some custom s-functions, (which is not specifically the case of test.mdl but it is indeed the normal case), then it is very important at this point to remember to copy the C code of every custom s-functions used in the scheme inside the generated scheme_rtai folder.

Another important thing to remember about s-functions is to always USE LOWERCASE characters in the file name, this is because the file names of the s-functions will be converted to lowercase within the make file (*.mk), which does not make any difference under windows but causes missing file errors under unix if the original name contained uppercase characters."

9. The code generation status is shown in the Matlab® command window.
10. The above procedure creates two directories in the actual Matlab® work directory. In this case "C:\...\My Documents\MATLAB\ejemplo-rtai ". The directories created are:

"test_rtai" and "slprj "

11. "test_rtai" contains the code generated by Matlab®/RTW® for "test.mdl"
12. In the PC (Host) copy on the PenDrive:

12.1. The directories placed in "C:\Program Files\MATLAB\R2011a "

- extern
- rtw
- simulink

12.2. The directory "test_rtai" placed in "C:\...\My Documents\Matlab\ejemplo-rtai ".

13. Remove the PenDrive from the PC (Host) and plug it into the CX-1020 unit (Target).
14. In the CX-1020 unit (Target) open a terminal, login as super-user and create the following directories:

```
# mkdir /usr/local/matlab
```

```
# mkdir /usr/local/src/models
```

15. Copy the below directories from the PenDrive:

```
extern, rtw, y simulink    →    /usr/local/matlab  
test_rtai                 →    /usr/local/src/models
```

16. Open a terminal, login as super-user and go to the directory “/usr/local/src/models/test_rtai”

```
# cd /usr/local/src/models/test_rtai
```

17. Compile the model:

```
# make -f test.mk
```

18. This procedure creates the executable file “test” into the root directory of “test_rtai” (in this case “models”).
Go to the directory “models”:

```
# cd /usr/local/src/models
```

19. Load the RTAI modules before executing the model:

```
# sync  
  
# insmod /usr/realtime/modules/rtai_hal.ko  
  
# insmod /usr/realtime/modules/rtai_lxrt.ko  
  
# insmod /usr/realtime/modules/rtai_fifos.ko  
  
# insmod /usr/realtime/modules/rtai_sem.ko  
  
# insmod /usr/realtime/modules/rtai_mbx.ko  
  
# insmod /usr/realtime/modules/rtai_msg.ko  
  
# insmod /usr/realtime/modules/rtai_netrpc.ko ThisNode="127.0.0.1"  
  
# sync
```

20. Execute the model. There are two ways to do it: Using “Direct Mode” (without graphical interface) and using RTAI-Lab (with graphical interface).

20.1. Option 1 – Direct Mode:

```
# ./test -v -f 5
```

(**Note:** 5 it's the execution time in seconds. Other values can be set)

The screen displays:

Target settings

```
=====
Real-time      :      HARD
Timing         :      internal / periodic
Priority        :      0
Finaltime      :      5.000000 [s]
CPU map        :      f
```

Target info

```
=====
Model name     :      test
Base sample time :      0.001000 [s]
Number of sample times :      1
Sample Time 0   :      0.001000 [s]
```

Target is running

Note: At the end of the execution time (5 [s]) the following messages are shown:

```
Final time occurred
Target is stopped
Target is terminated
```

This procedure shows the Target Settings, model information (name, sample time, etc.), and process state.

As the model doesn't have input/output signals associated, the information related to the I/O card "driver", such as state and signal specification (kind, range, etc), is not displayed.

20.2. Option 2 – RTAI-Lab:

Execute:

```
# ./test &
```

(The process starts in “wait” mode)

The screen displays:

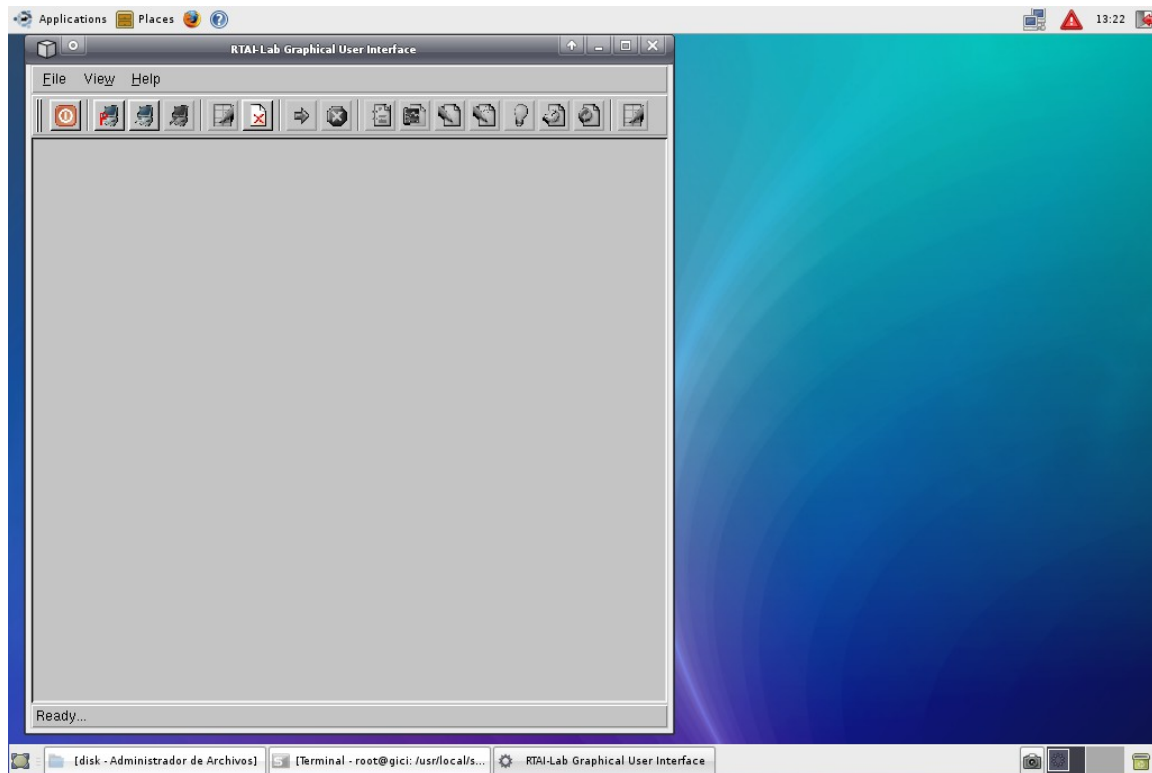
```
[1] 5267
```

(This number is the PID of “test”)

Start RTAI-Lab

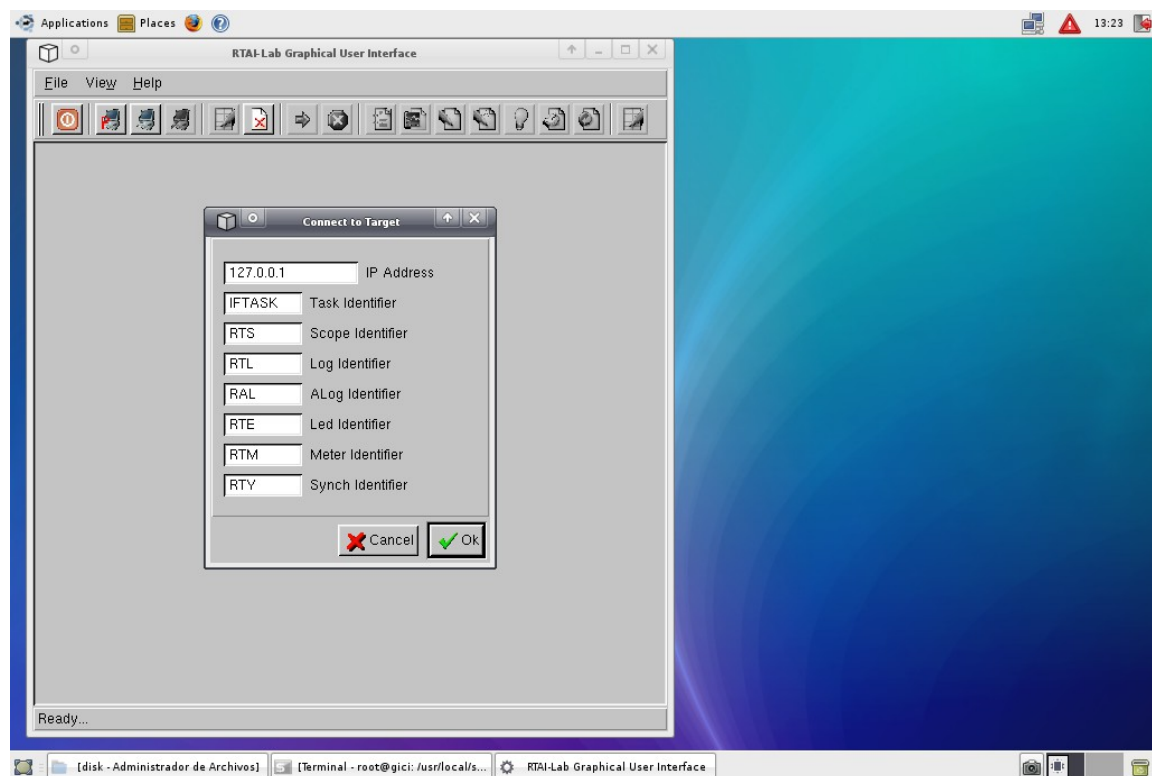
```
# /usr/realtime/bin/xrtailab &
```

Then, the graphical interface to RTAI-Lab is opened.

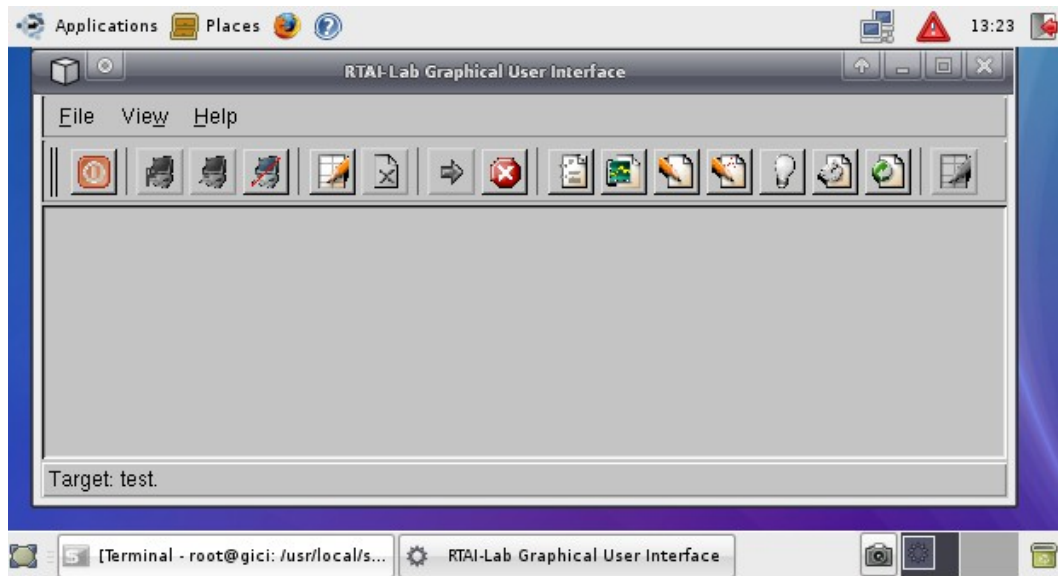


Connect to the “Target”.

Menu “File → Connect” shows the service identifiers between the “target” and RTAI-Lab (Don't modify). Click in “OK”.

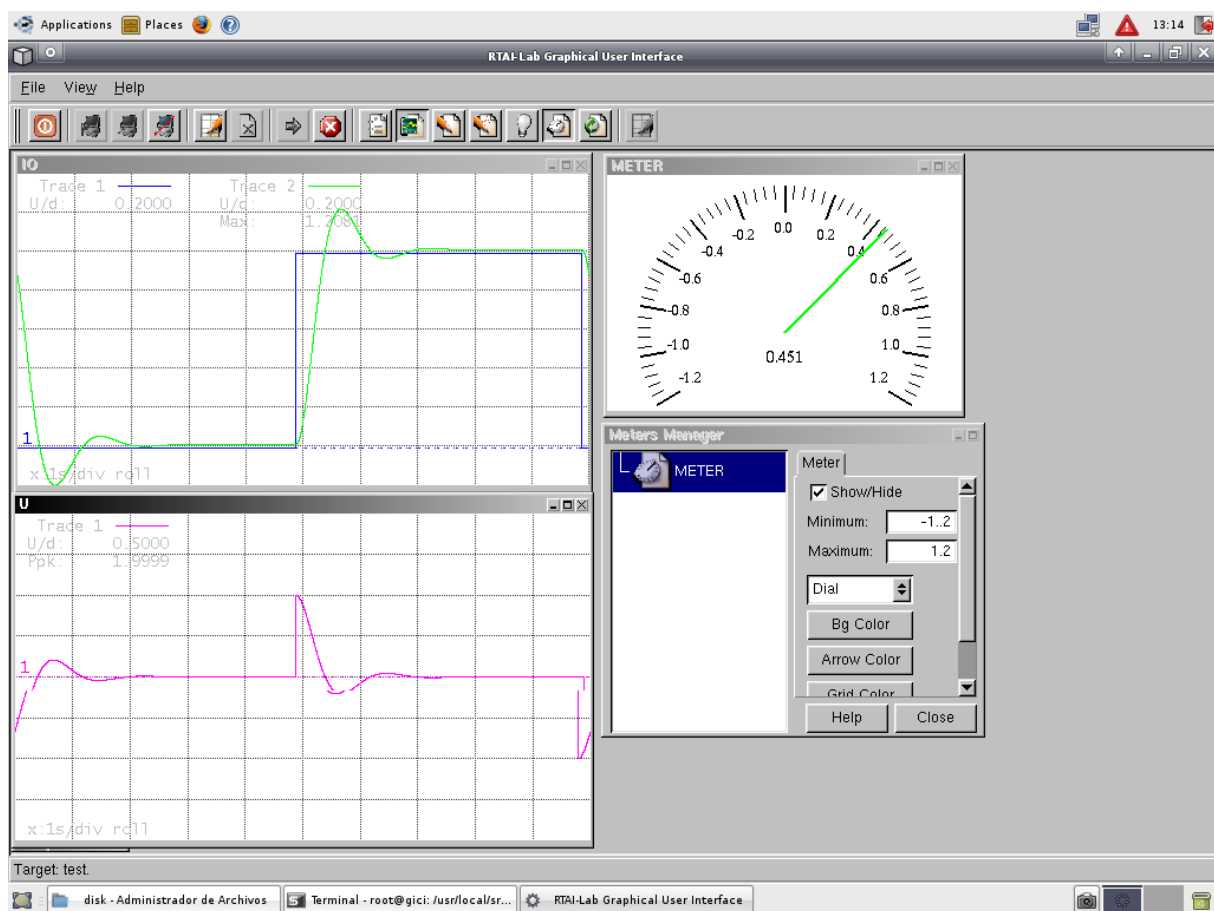


Once the target is connected, the buttons of the RTAI toolbar are enabled.



In the "View" menu the visualization elements are chosen and configured. In this case, "View → Scopes [OK]" enables the scopes. In the "Scope Manager" window the scale values (amplitude/time), offsets, color traces, etc. are set.

"View → Meters [OK]" enables the meter. The "Meter Manager" window enables the configuration parameters, as the "Scope Manager" do it.



The system behavior corresponds to the simulated in Matlab®/Simulink®.

To finalize, first stop the “Scopes“ and “Meters“, then disconnect the Target in “File→Disconnect“, finally “File → Quit “

21. Download the RTAI modules from the kernel:

```
# rmmod rtai_netrpc  
# rmmod rtai_msg  
# rmmod rtai_mbx  
# rmmod rtai_sem  
# rmmod rtai_fifos  
# rmmod rtai_lxrt  
# rmmod rtai_hal
```


AUTHOR'S NOTE

For any questions or comments contact:

- <http://gici.drupalgardens.com/contact>
- j.andres.cubillos@gmail.com

Any suggestion to improve this document will be appreciated.